

Università degli studi di Udine
Dipartimento di Scienze Matematiche, Fisiche, Informatiche
Corso di Laurea Magistrale in Informatica



RELAZIONE DEL PROGETTO PER IL CORSO DI *PROGRAMMAZIONE*
SU ARCHITETTURE PARALLELE

**SOLUZIONI PARALLELE DELL'ALGORITMO DI
ELIMINAZIONE DI GAUSS PER SISTEMI LINEARI
BOOLEANI CON OPERATORE XOR: IMPLEMENTAZIONE
E OTTIMIZZAZIONE**

Dipartimento di Scienze Matematiche, Fisiche, Informatiche

Bruno Ambrogio Innocente
Matricola n. 168381

Anno Accademico 2025/2026

Indice

1	Descrizione del problema affrontato	5
1.1	Algoritmo di eliminazione di Gauss	6
1.2	Variante algoritmo di Gauss per sistemi con coefficienti e variabili booleani e con operatore XOR	7
1.3	Esempi di esecuzione dell'algoritmo	9
1.3.1	Esempio 1: Sistema con soluzione	9
1.3.2	Esempio 2: Sistema irrisolvibile	10
2	Metodologia adottata per la soluzione	12
2.1	Versione seriale (<i>seriale.c</i>)	13
2.2	Versione p1 (<i>parallel1.cu</i>)	13
2.3	Versione p2 (<i>parallel2.cu</i>)	14
2.4	Versione p3 (<i>parallel3.cu</i>)	15
2.5	Versione p4 (<i>parallel4.cu</i>)	17
2.6	Versione p5 (<i>parallel5.cu</i>)	18
3	Risultati Sperimentali	20
3.1	Architetture utilizzate	20
3.2	Tempi di computazione	20
4	Conclusioni	29
4.1	Valutazioni e Osservazioni	29
4.2	Sviluppi futuri	30
A	Istruzioni di compilazione e guida all'uso	31
A.1	Guida all'uso del Makefile	31
A.1.1	Compilazione del progetto	31
A.1.2	Esecuzione delle versioni	32

A.1.3	Esecuzione su file di test	32
A.1.4	Pulizia dei file	33
A.2	File main.cu	33
A.3	file matrixgenerator.c	34
A.4	file Analysis.R	34
A.5	file SerialeDemo.c e p_demo.cu	34
A.6	directory result_data e test	34

Capitolo 1

Descrizione del problema affrontato

Lo scopo di questo progetto è quello di creare un programma che sfrutti le tecniche di programmazione parallela per risolvere un sistema di equazioni booleane a coefficienti booleani dove l'unica operazione ammessa è XOR ($\oplus$). Una possibile rappresentazione matematica del problema è la seguente:

$$\begin{cases} c_{1,1}x_1 \oplus c_{1,2}x_2 \oplus c_{1,3}x_3 \oplus \dots \oplus c_{1,(k-1)}x_{k-1} = t_1 \\ c_{2,1}x_1 \oplus c_{2,2}x_2 \oplus c_{2,3}x_3 \oplus \dots \oplus c_{2,(k-1)}x_{k-1} = t_2 \\ c_{3,1}x_1 \oplus c_{3,2}x_2 \oplus c_{3,3}x_3 \oplus \dots \oplus c_{3,(k-1)}x_{k-1} = t_3 \\ \vdots \\ c_{n,1}x_1 \oplus c_{n,2}x_2 \oplus c_{n,3}x_3 \oplus \dots \oplus c_{n,(k-1)}x_{k-1} = t_n \end{cases}$$

dove, per ogni $i = 1, \dots, n$ e $j = 1, \dots, k - 1$, $c_{i,j} \in \{0, 1\}$, $x_j \in \{0, 1\}$ e $t_i \in \{0, 1\}$

Un esempio concreto è il seguente:

$$\begin{cases} x_1 \oplus x_2 \oplus x_3 = 1 \\ x_1 \oplus x_2 \oplus x_4 = 1 \\ x_1 \oplus x_3 \oplus x_4 = 0 \\ x_2 \oplus x_3 \oplus x_4 = 1 \end{cases}$$

Che può essere riscritto in termini di matrice aumentata

$$\left[\begin{array}{cccc|c} C & \underline{t} \end{array} \right] = \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 \end{array} \right]$$

Dove C e $\underline{t}$ sono rispettivamente la matrice dei coefficienti e il vettore dei termini noti

1.1 Algoritmo di eliminazione di Gauss

Un metodo classico per risolvere sistemi di equazioni lineari, determinarne la compatibilità ed eventualmente calcolarne le soluzioni è l'algoritmo di eliminazione di Gauss.

Tale algoritmo si basa sulla trasformazione della matrice aumentata del sistema in una forma a scala per righe, mediante operazioni elementari che non alterano l'insieme delle soluzioni. Sia dato un sistema lineare rappresentato dalla matrice aumentata

$$\left[\begin{array}{c|c} C & \underline{t} \end{array} \right].$$

L'obiettivo è ottenere una matrice equivalente in cui le variabili possano essere determinate tramite un procedimento di sostituzione all'indietro.

Le operazioni elementari sulle righe utilizzate dall'algoritmo sono:

- scambio di due righe;
- moltiplicazione di una riga per uno scalare non nullo;
- somma di una riga con un multiplo di un'altra riga.

L'algoritmo procede colonna per colonna. Per ciascuna colonna si individua un elemento non nullo da utilizzare come pivot tra le righe non ancora considerate. Se necessario, si effettua uno scambio di righe per portare il pivot nella prima riga disponibile.

Sia r l'indice della prossima riga pivot. Ad ogni iterazione, una volta individuato un nuovo pivot, tale indice viene incrementato. Il valore di r al termine della procedura coincide con il rango della matrice, ovvero

il numero di pivot individuati che corrisponde anche al numero di righe linearmente indipendenti.

Una volta fissato il pivot, si procede ad annullare tutti gli elementi al di sotto di esso nella stessa colonna, mediante opportune combinazioni lineari delle righe. Questo processo viene ripetuto per tutte le colonne, ottenendo una matrice in forma a scala, in cui ogni pivot si trova a destra del pivot della riga precedente.

A differenza dell'algoritmo di Gauss-Jordan, in questa variante non vengono eliminati gli elementi al di sopra dei pivot, né è necessario ottenere una forma ridotta. Inoltre, la normalizzazione dei pivot non è strettamente richiesta ai fini della risoluzione del sistema.

Una volta ottenuta la forma a scala, le soluzioni del sistema vengono determinate tramite sostituzione all'indietro, risolvendo le equazioni a partire dall'ultima riga non nulla e procedendo verso l'alto.

In base alla forma finale della matrice, è possibile distinguere i seguenti casi:

- **Sistema determinato:** il numero di pivot è uguale al numero di variabili e il sistema ammette un'unica soluzione;
- **Sistema indeterminato:** il numero di pivot è inferiore al numero di variabili e il sistema ammette infinite soluzioni;
- **Sistema incompatibile:** esiste una riga della forma

$$[0 \ \dots \ 0 \mid t_j],$$

che rappresenta un'equazione impossibile, con $t_j \neq 0$.

1.2 Variante algoritmo di Gauss per sistemi con coefficienti e variabili booleani e con operatore XOR

Per il problema che questo progetto si pone di risolvere si considera una versione alternativa dell'algoritmo di Gauss classico per la risoluzione di sistemi di equazioni lineari. La variante considerata applica il metodo

di eliminazione a sistemi lineari definiti su variabili booleane, ovvero nel campo finito $(\mathbb{F}_2)$, in cui i coefficienti e le incognite assumono valori nell'insieme $(0,1)$. In questo contesto, le operazioni aritmetiche sono definite modulo 2: la somma corrisponde all'operatore XOR, mentre il prodotto corrisponde all'operatore AND.

La principale differenza rispetto all'algoritmo di Gauss classico riguarda il dominio di calcolo. Lavorando in $(\mathbb{F}_2)$, l'unico elemento invertibile è 1. Di conseguenza, la fase di normalizzazione del pivot non è necessaria, poiché ogni pivot è già uguale a 1.

L'algoritmo mantiene la struttura generale della ricerca dei pivot e dello scambio di righe, ma modifica il modo in cui vengono eseguite le operazioni di eliminazione. In particolare, l'eliminazione degli elementi sotto il pivot avviene mediante operazioni di XOR tra righe, che corrispondono alla somma modulo 2. Questo sostituisce completamente l'uso di combinazioni lineari con coefficienti arbitrari tipiche del caso reale.

Durante la fase di eliminazione, per ogni colonna si seleziona una riga contenente un valore non nullo da utilizzare come pivot. Se necessario, tale riga viene scambiata con la riga corrente (la riga con indice corrispondente al valore attuale del rango della matrice). Successivamente, tutte le righe sottostanti che presentano un valore non nullo nella colonna del pivot vengono aggiornate applicando l'operatore XOR tra la riga corrente e la riga pivot, in modo da annullare il coefficiente nella colonna considerata.

Terminata la fase di eliminazione, viene verificata la consistenza del sistema. In particolare, la presenza di una riga con tutti i coefficienti nulli ma termine noto uguale a 1 indica che il sistema è irrisolvibile.

Se il sistema risulta risolvibile, si procede con la sostituzione all'indietro. Per ciascuna riga, partendo dall'ultima contenente un pivot, si individua la variabile pivot e si assegna il suo valore iniziale pari al termine noto della riga. Successivamente, si aggiornano i valori tenendo conto delle variabili già determinate, utilizzando l'operatore AND per il prodotto e XOR per la somma. Questo passaggio equivale allo spostamento dei termini noti da un lato all'altro dell'equazione nel caso classico, ma nel contesto booleano somma e sottrazione coincidono.

In sintesi, le differenze principali rispetto all'algoritmo di Gauss clas-

sico sono: l'utilizzo del campo finito ($\mathbb{F}_2$) al posto dei numeri reali, la sostituzione delle operazioni aritmetiche standard con XOR e AND, e l'assenza della fase di normalizzazione del pivot. La rappresentazione in pseudocodice dell'algoritmo è riportata nell'Algoritmo 1.

Un altro aspetto da tenere a mente è che l'algoritmo che si andrà ad implementare deve avere queste caratteristiche: determinare se il sistema ha almeno una soluzione e, nel caso, calcolare la soluzione o una qualsiasi nel caso di più soluzioni.

1.3 Esempi di esecuzione dell'algoritmo

Di seguito vengono riportati degli esempi di dimensioni ridotte riguardanti il funzionamento dell'algoritmo di Gauss per i sistemi booleani considerati. La notazione E_{ij} rappresenta un'operazione di somma (XOR): alla riga i viene sommata la riga j . La notazione S_{ij} rappresenta un'operazione di scambio di righe: significa che la riga i viene scambiata con la riga j .

1.3.1 Esempio 1: Sistema con soluzione

Dato il seguente sistema

$$\begin{cases} x_1 \oplus x_2 \oplus x_3 = 1 \\ x_1 \oplus x_2 \oplus x_4 = 1 \\ x_1 \oplus x_3 \oplus x_4 = 0 \\ x_2 \oplus x_3 \oplus x_4 = 1 \end{cases}$$

Si riscrive in termini di matrice aumentata e si procede con l'applicazione dell'algoritmo di Gauss

$$\left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 \end{array} \right] \xrightarrow{E_{2,1}; E_{3,1}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{array} \right] \xrightarrow{S_{2,3}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \end{array} \right] \xrightarrow{E_{4,2}}$$

$$\xrightarrow{E_{4,2}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{array} \right] \xrightarrow{E_{4,3}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{array} \right]$$

In questo caso il sistema ammette più di una soluzione (la riga 4 è libera). Fissando $x_4 = 0$ e procedendo con la sostituzione all'indietro una possibile soluzione è $x_1 = 0; x_2 = 1; x_3 = 0; x_4 = 0$.

1.3.2 Esempio 2: Sistema irrisolvibile

Dato il seguente sistema

$$\begin{cases} x_1 \oplus x_2 \oplus x_3 = 1 \\ x_1 \oplus x_2 \oplus x_4 = 0 \\ x_2 \oplus x_3 \oplus x_4 = 1 \\ x_1 \oplus x_3 \oplus x_4 = 0 \\ x_1 \oplus x_2 \oplus x_3 \oplus x_4 = 1 \end{cases}$$

Si riscrive in termini di matrice aumentata e si procede con l'applicazione dell'algoritmo di Gauss

$$\begin{aligned} & \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{array} \right] \xrightarrow{E_{2,1}; E_{4,1}; E_{5,1}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{array} \right] \xrightarrow{S_{2,3}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{array} \right] \xrightarrow{E_{4,2}} \\ & \xrightarrow{E_{4,2}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{array} \right] \xrightarrow{E_{4,3}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{array} \right] \xrightarrow{E_{5,4}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{array} \right] \end{aligned}$$

Essendo presente una riga con coefficienti tutti 0 e termine noto pari a 1 si conclude che questo sistema non ha soluzione (irrisolvibile).

Algorithm 1: Pseudocodice per l'implementazione dell'algoritmo seriale di Gauss per la risoluzione di sistemi di equazioni lineari booleani

Data: A matrice $n \times k$; *solution* vettore $1 \times (k - 1)$

```

1   $rank \leftarrow 0$ ;  $pivot \leftarrow -1$ ;  $pivCol \leftarrow -1$ ;
2  for  $z = 1$  to  $k - 1$  do // Inizializzazione vettore soluzioni
3       $solution[z] = 0$ ;
4  for  $j = 0$  to  $k - 1$  and  $rank < n$  do
5       $pivot \leftarrow -1$ ;
6      for  $row = rank$  to  $n$  do // cerca il pivot
7          if  $A_{row,col}$  then
8               $pivot = row$ ;
9              break;
10     if  $pivot == -1$  then
11         continue // colonna nulla
12     if  $pivot \neq rank$  then
13          $SwapRows(pivotRow, RankRow)$ ;
14     for  $i = rank + 1$  to  $n$  do
15         if  $A_{i,j} == 1$  then
16              $A_{i,j} = A_{i,j} XOR A_{rank,j}$ ; // rows elimination
17      $rank \leftarrow rank + 1$ ;
18 // controllo se il sistema ha almeno una soluzione
19 for  $i = rank$  to  $n$  do
20     if  $A_{i,k-1} == 1$  then
21         return false;
22 for  $i = rank - 1$  to  $i \geq 0$  do /* Back substitution, ciclo
    For decrescente */
23     for  $m = 0$  to  $m < k$  do
24         if  $A_{i,m}$  then
25              $pivCol = m$ ;
26             break;
27      $solution[pivCol] = A_{i,k-1}$ ;
28     for  $int j = pivCol + 1$  to  $j < k-1$  do
29          $solution[pivCol] =$ 
             $solution[pivCol] XOR (A_{i,j} AND solution[j])$ ;
30 return true;

```

Capitolo 2

Metodologia adottata per la soluzione

Per questo progetto sono state implementate diverse versioni parallele dell'algoritmo di Gauss per sistemi booleani. In questo capitolo viene descritto e spiegato il funzionamento di ogni variante implementata dell'algoritmo. Prima di descrivere ogni versione, è utile schematizzare l'algoritmo come segue:

- **Loop principale:**
 1. **ricerca pivot**;
 2. eventuale **scambio di righe**;
 3. **eliminazione righe** (con valore pari ad 1 nella cella corrispondente alla colonna del pivot);
- **controllo risolubilità del sistema**;
- **sostituzione all'indietro**, in presenza di una o più soluzioni.

Questa suddivisione fornisce uno schema mentale semplice per spiegare successivamente le diverse strategie implementate. Per ogni versione implementata, se l'algoritmo trova una soluzione restituisce valore **true** e scrive una possibile soluzione nel vettore passato alla funzione. Se il sistema viene determinato irrisolvibile il vettore delle soluzioni rimane inizializzato a zero e viene restituito il valore **false**.

2.1 Versione seriale (*seriale.c*)

Il funzionamento dell'algoritmo seriale è già descritto nel capitolo 1. Questo algoritmo è implementato in un programma C. Questa versione funge da benchmark per confrontare le prestazioni delle versioni parallele e validare la correttezza delle soluzioni ottenute. La matrice dei dati in questa soluzione viene implementata come matrice di dati booleani (array dinamico di n puntatori, dove ogni puntatore punta ad un array dinamico di k booleani). Tutte le versioni parallele usano blocchi con 256 thread.

2.2 Versione p1 (*parallel1.cu*)

La versione p1 rappresenta un primo tentativo di parallelizzazione "dummy" dell'algoritmo di eliminazione di Gauss per sistemi booleani, realizzata mediante CUDA. In questa implementazione, il parallelismo viene introdotto esclusivamente nella fase di eliminazione delle righe, mentre le restanti parti dell'algoritmo (ricerca del pivot, scambio di righe e sostituzione all'indietro) rimangono eseguite sulla CPU.

L'algoritmo mantiene la stessa struttura logica della versione seriale:

- ricerca del pivot (CPU)
- eventuale scambio di righe (CPU)
- eliminazione delle righe sotto il pivot (GPU)
- controllo di consistenza del sistema (CPU)
- *back-substitution* (CPU)

La scelta progettuale principale consiste quindi nel parallelizzare il passo più costoso computazionalmente, ovvero l'eliminazione delle righe.

La matrice aumentata viene rappresentata come un array lineare di elementi di tipo `uint8_t`, contenente valori booleani (0 o 1). La matrice è memorizzata in memoria contigua in formato *row-major*.

Una copia della matrice viene allocata anche sulla GPU (`d_matrix`), sulla quale vengono eseguite le operazioni parallele.

Il kernel principale è `rowsElimination`, in cui ogni thread è responsabile dell'elaborazione di una o più righe. Viene utilizzata una strategia a stride (*grid-stride loops*) per coprire tutte le righe e rendere la procedura scalabile per matrici grandi. L'operazione di eliminazione è quindi completamente parallelizzata sulle righe, sfruttando il fatto che queste operazioni (per lo stesso pivot) sono indipendenti tra loro.

Ad ogni iterazione del *loop* principale:

- la matrice viene copiata dalla CPU alla GPU (solo in caso di *swap*);
- il kernel viene eseguito sulla GPU;
- il risultato viene copiato nuovamente dalla GPU alla CPU.

Questa scelta semplifica l'implementazione, ma introduce un overhead dovuto ai trasferimenti di memoria host-device. Questa versione rappresenta quindi una soluzione semplice per introdurre il parallelismo, evidenziando però il limite dovuto al costo delle copie di memoria.

2.3 Versione p2 (*parallel2.cu*)

La versione p2 introduce un'ottimizzazione significativa rispetto alla precedente, migliorando l'efficienza dell'accesso ai dati tramite una rappresentazione compatta della matrice. La struttura generale dell'algoritmo rimane invariata rispetto alla versione p1. La principale differenza riguarda la rappresentazione dei dati e il modo in cui vengono eseguite le operazioni di eliminazione. In questa versione, la matrice non è più memorizzata come array di byte, ma come array di interi a 32 bit (`uint32_t`), dove ogni bit rappresenta un coefficiente booleano. Nello specifico, ogni riga è suddivisa in *word* da 32 bit. Quindi più coefficienti vengono compressi in una singola *word*. Il numero di *word* per riga è dato da:

$$numWords = \text{ceil}(k/32)$$

Per accedere ai singoli elementi è stata implementata una funzione dedicata (`getBit`).

Il kernel `rowsElimination2` definito in questo programma segue lo stesso schema della versione precedente, ma opera a livello di *word* invece che di singolo elemento.

Per ogni riga: si verifica se il bit corrispondente alla colonna del pivot è 1 se sì, si esegue XOR tra intere *word* della riga e quelle della riga pivot. Questo permette di effettuare 32 operazioni logiche in parallelo con una singola istruzione.

Per quanto concerne l'accesso ai dati, anche in questo caso, la matrice viene copiata tra host e device ad ogni iterazione e il kernel opera sulla versione residente in GPU. Tuttavia, grazie alla rappresentazione compatta, il volume di dati trasferiti è ridotto e l'accesso in memoria globale è più efficiente.

2.4 Versione p3 (*parallel3.cu*)

La versione p3 rappresenta un'evoluzione significativa delle versioni precedenti, in cui l'obiettivo principale è ridurre drasticamente l'overhead dovuto ai trasferimenti di memoria tra CPU e GPU e aumentare il grado di parallelismo complessivo. A differenza delle versioni p1 e p2, in cui solo la fase di eliminazione veniva eseguita sulla GPU, in questa implementazione vengono parallelizzate:

- ricerca del pivot;
- scambio di righe;
- eliminazione.

L'intero ciclo principale dell'algoritmo viene quindi eseguito sulla GPU, mentre la CPU si occupa solo di:

- inizializzazione dei dati;
- copia iniziale e finale della matrice;
- controllo di consistenza;
- *back-substitution*.

La matrice è rappresentata utilizzando la stessa tecnica di bit-packing introdotta nella versione p2: il tipo di dato è `uint32_t`, ogni bit della *word* rappresenta un coefficiente booleano e le righe sono memorizzate come array di *word* da 32 bit. Questo consente di sfruttare operazioni bitwise per eseguire più operazioni logiche in parallelo.

La versione p3 introduce tre kernel principali:

1. Ricerca del pivot (**findPivotKernel**)

La ricerca del pivot viene eseguita in parallelo. Ogni thread analizza un sottoinsieme delle righe e se trova un elemento valido (bit = 1 nella colonna corrente) aggiorna il pivot globale tramite `atomicMin`. L'uso di `atomicMin` garantisce che venga selezionata la riga con indice minimo, mantenendo la correttezza dell'algoritmo.

2. Scambio di righe (**swapRowsKernel**)

Lo scambio di righe viene parallelizzato a livello di *word*: ogni thread scambia una o più *word* delle due righe e l'operazione è indipendente per ogni *word*. Questo approccio migliora notevolmente le prestazioni rispetto allo *swap* sequenziale.

3. Eliminazione (**eliminationKernel**)

La fase di eliminazione segue lo stesso schema della versione p2: per ogni riga sotto il pivot, se il bit nella colonna del pivot è 1, viene eseguito XOR tra tutte le *word* della riga e quelle della riga pivot.

La differenza più importante rispetto alle versioni precedenti riguarda la gestione della memoria: la matrice viene copiata sulla GPU una sola volta all'inizio. Tutte le operazioni principali avvengono su `d_matrix` (matrice in memoria globale sulla GPU) e il risultato viene copiato sulla CPU una sola volta alla fine. Questo elimina completamente il costo di copiare tutta la matrice ad ogni iterazione, che rappresentava un forte limite in p1 e p2.

La versione p3 rappresenta un buon compromesso tra complessità implementativa e prestazioni. L'algoritmo sfrutta in modo più completo le capacità della GPU, riducendo i costi di comunicazione e aumentando il parallelismo disponibile.

2.5 Versione p4 (*parallel4.cu*)

La versione p4 introduce un approccio radicalmente diverso rispetto alle versioni precedenti, sfruttando il *dynamic parallelism* di CUDA e, in particolare, il meccanismo di esecuzione sequenziale tramite `cudaStreamTailLaunch`. L'obiettivo principale di questa versione è eliminare completamente il controllo iterativo lato CPU, spostando l'intero flusso dell'algoritmo sulla GPU. A differenza delle versioni precedenti, in cui la CPU gestiva il ciclo principale sulle colonne, in questa implementazione: il ciclo dell'algoritmo viene avviato dalla CPU, ma l'intera sequenza di operazioni (ricerca pivot $\rightarrow$ swap $\rightarrow$ eliminazione $\rightarrow$ aggiornamento *rank*) viene eseguita direttamente sulla GPU. I kernel CUDA lanciano altri kernel in modo sequenziale. Il punto chiave di questa versione è l'uso di kernel che lanciano altri kernel. L'esecuzione sequenziale è garantita tramite `cudaStreamTailLaunch`. In particolare: `findPivotKernel4` lancia `swapRowsKernel4`, che a sua volta lancia `eliminationKernel4`, che a sua volta lancia `increaseRank4`.

L'utilizzo di `cudaStreamTailLaunch` garantisce che i kernel vengano eseguiti in ordine sequenziale, senza bisogno di sincronizzazione esplicita lato CPU. Questo permette di costruire una vera e propria *pipeline* completamente residente su GPU.

Per quanto riguarda la rappresentazione dei dati si continua come nelle versioni p2 e p3: la matrice è rappresentata tramite `uint32_t`, utilizzo di bit-packing e accesso ai coefficienti tramite operazioni bitwise. Inoltre, variabili globali come il pivot e il *rank* sono memorizzate in memoria globale della GPU (`d_pivot`, `d_rank`) e condivise tra i kernel.

Il flusso diventa quindi:

1. Reset e avvio (`resetPivot4`): inizializza il pivot a un valore sentinella e dopo lancia il kernel di ricerca del pivot;
2. Ricerca del pivot (`findPivotKernel4`): la parallelizzazione è sulle righe. Il kernel utilizza `atomicMin` per trovare il pivot minimo, e, al termine, lancia il kernel di swap;

3. Scambio di righe (`swapRowsKernel4`): la parallelizzazione è sulle *word*. Se il pivot è valido, effettua lo scambio, e, successivamente lancia il kernel di eliminazione;
4. Eliminazione (`eliminationKernel4`): la parallelizzazione è sulle righe. Il kernel esegue XOR tra righe come nelle versioni precedenti, e, al termine, aggiorna il rango tramite un kernel separato;
5. Aggiornamento rango (`increaseRank4`).

Questa implementazione è particolarmente interessante, in quanto dimostra come sia possibile spostare completamente il controllo dell'algoritmo sulla GPU.

2.6 Versione p5 (*parallel5.cu*)

La versione p5 estende la versione p3 introducendo un uso esplicito della *shared memory* della GPU, con l'obiettivo di ridurre ulteriormente il numero di accessi alla memoria globale e migliorare le prestazioni della fase di eliminazione.

La struttura generale dell'algoritmo rimane invariata rispetto alla versione p3:

- ricerca del pivot (GPU)
- scambio di righe (GPU)
- eliminazione (GPU)
- copia finale dei dati (GPU $\rightarrow$ CPU)
- controllo di consistenza (CPU)
- *back-substitution* (CPU)

Anche in questa versione, la matrice viene copiata sulla GPU una sola volta all'inizio e riportata sulla CPU solo al termine dell'elaborazione. La rappresentazione della matrice è identica alla versione p2 e p3. La versione p5 mantiene i kernel della versione p3 per ricerca del pivot (`findPivotKernel5`) e lo scambio di righe (`swapRowsKernel5`). Tuttavia,

introduce una versione ottimizzata del kernel di eliminazione. Questo, utilizza la *shared memory* per memorizzare temporaneamente la riga pivot. Questa viene caricata in modo cooperativo dai thread del blocco in un array condiviso (`s_pivot`). Viene utilizzata la sincronizzazione (`__syncthreads`) per garantire che tutti i dati siano disponibili. Successivamente, blocco viene associato ad una riga della matrice e un thread per ogni blocco verifica se la riga deve essere aggiornata (`bit pivot = 1`). Il risultato viene condiviso tra i thread tramite una variabile condivisa (`active`). Se la riga deve essere "eliminata", ogni thread esegue l'operazione XOR su una *word* (o più *word* in caso di matrici con molte colonne tramite *grid-stride loops*)

Rispetto alla versione p3, il miglioramento principale riguarda il pattern di accesso alla memoria: la riga pivot viene letta dalla memoria globale una sola volta per blocco. Le operazioni successive utilizzano la *shared memory* in modo da ridurre significativamente il numero di accessi ripetuti alla memoria globale.

Capitolo 3

Risultati Sperimentali

3.1 Architetture utilizzate

Per la valutazione sperimentale delle diverse versioni dell'algoritmo sono state utilizzate due differenti configurazioni hardware, indicate nel seguito come Configurazione 1 e Configurazione 2.

I test sono stati eseguiti su sistema operativo Linux in ambiente WSL: nello specifico, Ubuntu 22.04.5 LTS su Windows 11 per la Configurazione 1 e Ubuntu 24.04.4 LTS su Windows 10 per la Configurazione 2. Le versioni del toolkit CUDA utilizzate sono rispettivamente la 13.1 e la 12.0.

Le caratteristiche principali delle due configurazioni sono riportate nelle Tabelle [3.1](#) e [3.2](#).

Le due configurazioni differiscono principalmente per capacità computazionale della GPU e dimensione della memoria video, fattori che influenzano significativamente le prestazioni delle versioni parallele.

3.2 Tempi di computazione

Per l'analisi delle prestazioni sono state utilizzate matrici quadrate di dimensione $n \times n$ con valori di n crescenti a passi di 512 elementi.

Per ciascuna dimensione della matrice, ogni test è stato ripetuto dieci volte, registrando il tempo medio di esecuzione al fine di ridurre la variabilità dovuta a fattori esterni.

	Configurazione 1	
	CPU	GPU
	AMD Ryzen 7 5800H	NVIDIA GeForce RTX 3050 Ti Laptop GPU
Cores	8	2560
Clock base/max	3.20 GHz/4.45 GHz	1.22 GHz/1.49 GHz
Memoria	16 GB	4 GB
Tipo di memoria	DDR4	GDDR6

Tabella 3.1: Specifiche hardware della Configurazione 1

	Configurazione 2	
	CPU	GPU
	Intel Core i7 5820K	NVIDIA GeForce RTX 3060
Cores	6	3584
Clock base/max	3.30 GHz/ 4.30 GHz	1.32 GHz/1.90 GHz
Memoria	16GB	12GB
Tipo di memoria	DDR4	GDDR6

Tabella 3.2: Specifiche hardware della Configurazione 2

Le matrici sono state generate casualmente tramite un apposito programma (*matrixgenerator.c*), che produce valori booleani (0 o 1) in funzione del parametro θ (**theta** nel codice). Tale parametro rappresenta la probabilità che un elemento della matrice assuma valore 1. Di conseguenza, $1 - \theta$ approssima il grado di sparsità della matrice. La generazione è stata fatta fissando un *seed* in modo da poter comparare adeguatamente i risultati delle diverse versioni e configurazioni.

I test sono stati eseguiti per i seguenti valori di θ

$$0.1, 0.3, 0.5, 0.7, 0.9$$

Per ogni valore di n , è stato infine considerato il tempo medio complessivo sui diversi valori di θ .

Le Figure 3.1 e 3.2 mostrano il confronto tra i tempi medi di esecuzione delle diverse versioni dell'algoritmo sulla configurazione 2. In particolare:

la Figura 3.1 riporta i tempi medi considerando tutti i valori di θ . La Figura 3.2 mostra l'andamento delle prestazioni separatamente per ciascun valore di θ .

Dall'analisi dei risultati emerge chiaramente che, all'aumentare della dimensione della matrice, le versioni p3, p4 e p5 risultano le più efficienti.

La versione seriale è la meno performante, seguita dalla versione p1, che introduce un parallelismo limitato. Entrambe sono incluse principalmente a scopo di confronto e validazione.

La versione p2 rappresenta un miglioramento significativo rispetto alle precedenti, grazie alla rappresentazione compatta dei dati, ma non raggiunge le prestazioni delle versioni più avanzate.

Le Figure 3.3 e 3.4 forniscono un'analisi più dettagliata delle tre versioni migliori (p3, p4 e p5), mostrando uno “zoom” sui tempi di esecuzione e permettendo un confronto più accurato.

Inoltre, le Figure 3.5 e 3.6 riportano risultati analoghi ottenuti sulla Configurazione 1. In questo caso si osserva un comportamento leggermente differente: in particolare, la versione p5 risulta generalmente più efficiente della p4 anche su matrici di dimensione più contenuta, a differenza di quanto osservato nella Configurazione 2, evidenziando come le ottimizzazioni basate su *shared memory* possano essere particolarmente efficaci su architetture con minori risorse GPU. I risultati dei tempi medi per ogni versione sono riportati nelle tabelle 3.3, 3.4 e 3.5. La tabella 3.5 mostra i risultati aggiuntivi delle versioni p3, p4 e p5 da $n = 10752$ in pa $n = 20480$ per la configurazione 2.

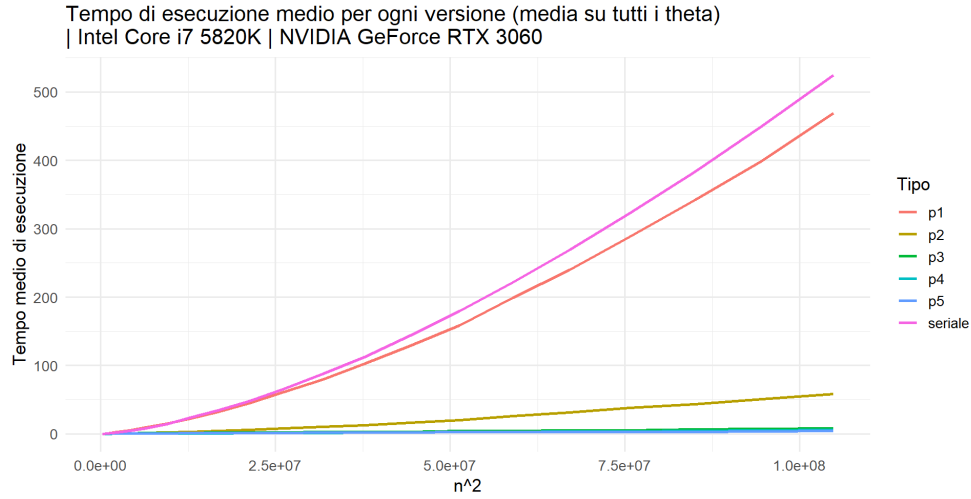


Figura 3.1: Tempi di esecuzione medi di tutte le versioni implementate al variare della dimensione della matrice (n^2), media su tutti i valori di θ . Configurazione 2. Dimensione massima: $n = 10240$.

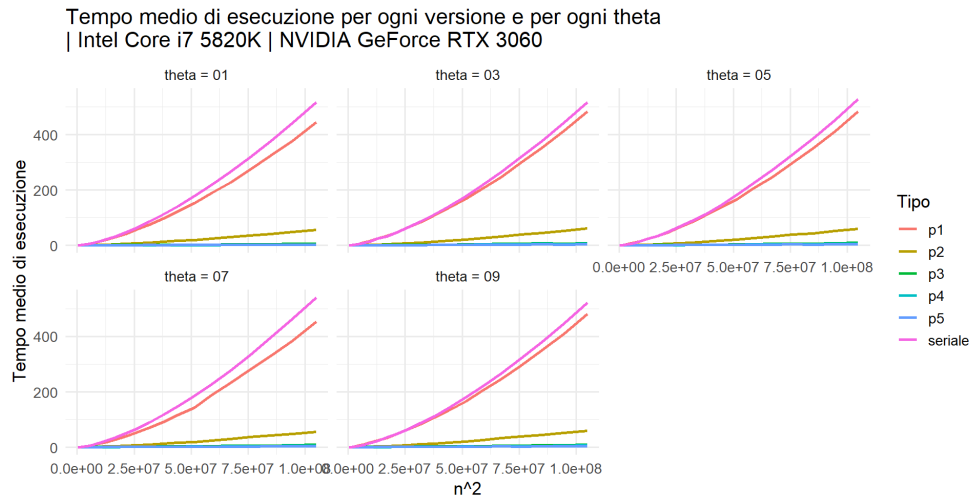


Figura 3.2: Tempi di esecuzione medi di tutte le versioni implementate al variare della dimensione della matrice (n^2), suddivisi per valore di θ . Configurazione 2. Dimensione massima: $n = 10240$.

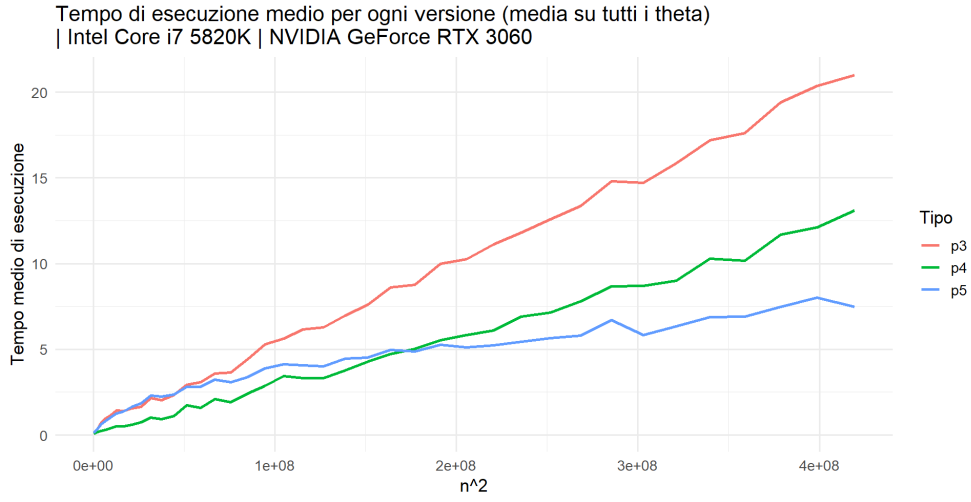


Figura 3.3: Tempi di esecuzione medi delle versioni p3, p4 e p5 al variare della dimensione della matrice (n^2), mediati su tutti i valori di θ . Configurazione 2. Dimensione massima: $n = 20480$

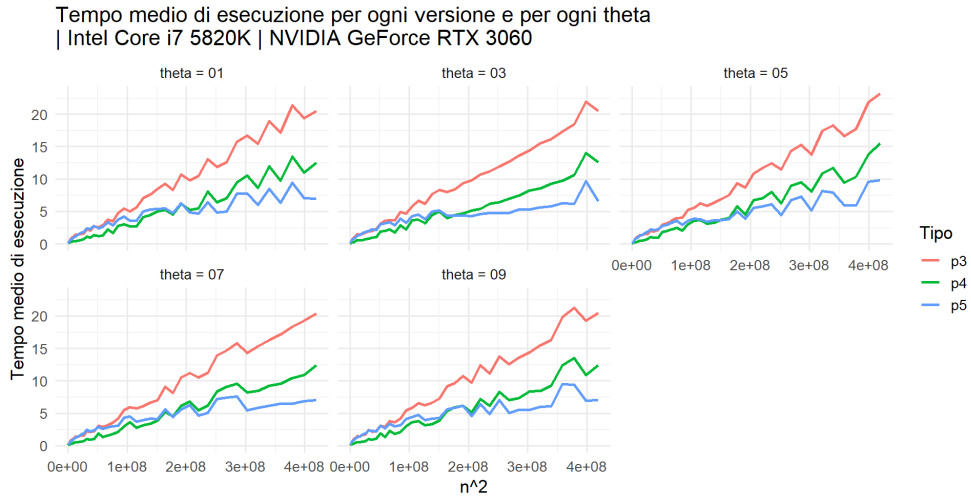


Figura 3.4: Tempi di esecuzione medi delle versioni p3, p4 e p5 al variare della dimensione della matrice (n^2), suddivisi per valore di θ . Configurazione 2. Dimensione massima: $n = 20480$

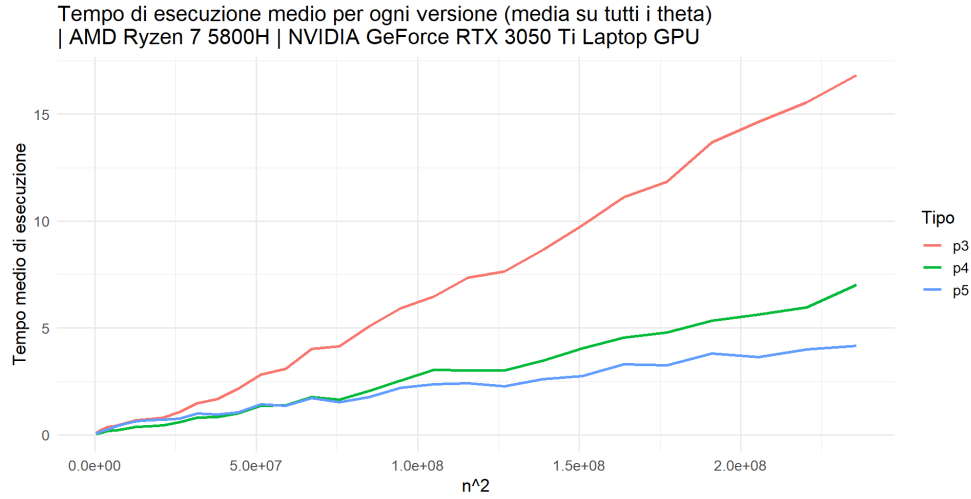


Figura 3.5: Tempi di esecuzione medi delle versioni p3, p4 e p5 al variare della dimensione della matrice (n^2), mediati su tutti i valori di θ . Configurazione 1. Dimensione massima: $n = 15360$.

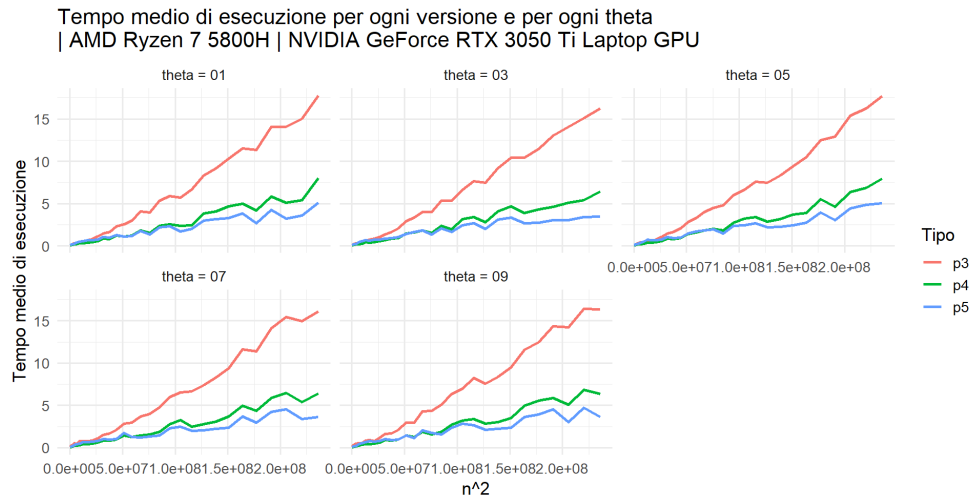


Figura 3.6: Tempi di esecuzione medi delle versioni p3, p4 e p5 al variare della dimensione della matrice (n^2), suddivisi per valore di θ . Configurazione 1. Dimensione massima: $n = 15360$

Tabella 3.3: Tempo medio di esecuzione per ogni versione (media su tutti i θ). Configurazione 2: Intel Core i7 5820K, NVIDIA GeForce RTX 3060. Valori di n da 512 a 10240

n	p1	p2	p3	p4	p5	seriale
512	0.1842	0.0916	0.1457	0.0700	0.1266	0.0806
1024	0.9075	0.2042	0.1950	0.1121	0.2059	0.5704
1536	2.4696	0.4553	0.3549	0.1791	0.3518	1.9196
2048	4.8831	0.8893	0.7139	0.2474	0.6223	4.3383
2560	9.3106	1.5754	0.9509	0.2986	0.8042	8.2882
3072	14.9773	2.3334	1.1377	0.3892	1.0129	14.3204
3584	22.4423	3.1698	1.4226	0.5118	1.2586	23.4725
4096	32.1191	4.2615	1.4136	0.5227	1.3847	34.3066
4608	44.8991	5.7777	1.5561	0.5954	1.6541	47.9487
5120	61.2584	8.0297	1.6301	0.7403	1.8559	65.6037
5632	79.3727	10.3809	2.1454	1.0045	2.2910	87.2664
6144	102.2454	12.8178	2.0408	0.9411	2.2327	112.6612
6656	128.6911	16.5306	2.3444	1.0940	2.3759	143.6767
7168	158.7787	20.2638	2.9360	1.7471	2.8245	180.0598
7680	199.1597	26.1491	3.0880	1.5822	2.8028	222.1244
8192	239.9990	31.3628	3.5896	2.0974	3.2413	269.1805
8704	288.2113	37.8475	3.6544	1.9013	3.0730	322.9355
9216	341.5720	43.6285	4.4185	2.4086	3.3865	382.8521
9728	399.8254	50.9539	5.2771	2.8659	3.8951	450.2172
10240	469.3043	58.5142	5.6271	3.4387	4.1324	524.9335

Tabella 3.4: Tempo medio di esecuzione in secondi per versioni p3, p4 e p5 (media su tutti i θ). Configurazione 1: AMD Ryzen 7 5800H, NVIDIA GeForce RTX 3050 Ti Laptop GPU. Valori di n da 512 a 15360

n	p3	p4	p5
512	0.0939	0.0561	0.0707
1024	0.1700	0.0819	0.1215
1536	0.2606	0.1280	0.1868
2048	0.3805	0.1925	0.2636
2560	0.4377	0.2227	0.4111
3072	0.5552	0.2991	0.5244
3584	0.6890	0.3883	0.6563
4096	0.7449	0.4033	0.6880
4608	0.8112	0.4608	0.7142
5120	1.0897	0.5918	0.7641
5632	1.4826	0.8270	1.0043
6144	1.6765	0.8377	0.9497
6656	2.1465	0.9966	1.0563
7168	2.8248	1.3751	1.4325
7680	3.1044	1.3833	1.3747
8192	4.0255	1.7658	1.7240
8704	4.1586	1.6551	1.5340
9216	5.0822	2.0688	1.7624
9728	5.9120	2.5420	2.1982
10240	6.4819	3.0414	2.3812
10752	7.3564	3.0100	2.4186
11264	7.6509	3.0148	2.2749
11776	8.6541	3.4878	2.6111
12288	9.8079	4.0581	2.7549
12800	11.1294	4.5469	3.3011
13312	11.8396	4.7873	3.2615
13824	13.6990	5.3580	3.8156
14336	14.6572	5.6370	3.6466
14848	15.5616	5.9720	4.0001
15360	16.8385	7.0202	4.1734

Tabella 3.5: Tempo medio di esecuzione per ogni versione (media su tutti i θ). Configurazione 2: Intel Core i7 5820K, NVIDIA GeForce RTX 3060. Valori di n da 10752 a 20480.

n	p3	p4	p5
10752	6.1714	3.3143	4.0734
11264	6.2793	3.3299	4.0183
11776	6.9744	3.7679	4.4539
12288	7.5792	4.2819	4.5060
12800	8.6055	4.7376	4.9640
13312	8.7559	5.0255	4.8612
13824	9.9870	5.5255	5.2538
14336	10.2629	5.8228	5.0991
14848	11.1286	6.1109	5.2286
15360	11.8012	6.9154	5.4365
15872	12.5804	7.1487	5.6416
16384	13.3709	7.8144	5.7992
16896	14.8070	8.6672	6.6883
17408	14.7176	8.6911	5.8274
17920	15.8364	8.9935	6.3477
18432	17.1763	10.2870	6.8885
18944	17.6062	10.1660	6.9110
19456	19.4132	11.6781	7.4791
19968	20.3598	12.1196	8.0048
20480	20.9954	13.0851	7.4715

Capitolo 4

Conclusioni

4.1 Valutazioni e Osservazioni

In questo progetto è stato affrontato il problema della risoluzione di sistemi lineari booleani tramite l'algoritmo di eliminazione di Gauss, con l'obiettivo di migliorarne le prestazioni attraverso tecniche di parallelizzazione su GPU.

Sono state sviluppate diverse versioni parallele dell'algoritmo, caratterizzate da un livello crescente di complessità e ottimizzazione. In particolare:

- la versione p1 introduce una prima forma di parallelismo limitata alla fase di eliminazione;
- la versione p2 migliora l'efficienza tramite una rappresentazione compatta dei dati (*bit-packing*);
- la versione p3 estende il parallelismo a tutte le fasi principali, riducendo significativamente l'overhead di comunicazione;
- la versione p4 esplora l'uso del *dynamic parallelism*, spostando il controllo dell'algoritmo sulla GPU;
- la versione p5 introduce ottimizzazioni avanzate basate su *shared memory*.

Dai risultati sperimentali emerge come le versioni più ottimizzate (in particolare p3 e p5) offrano un miglioramento significativo delle prestazioni rispetto alla versione seriale, grazie a una migliore gestione della memoria e a un più elevato grado di parallelismo. La versione migliore dal punto di vista di efficienza misurata come tempo d'esecuzione è la p5. La versione p4 è inizialmente più efficiente perché evita sincronizzazioni CPU-GPU grazie al *kernel chaining*, riducendo l'overhead per problemi piccoli. Tuttavia, al crescere della dimensione della matrice, p5 diventa più efficiente grazie a un migliore utilizzo della memoria (*shared memory*) e parallelismo più strutturato, che riducono il costo computazionale dominante.

4.2 Sviluppi futuri

Sono possibili diverse estensioni e miglioramenti del lavoro svolto.

Un ulteriore sviluppo potrebbe riguardare l'introduzione di una parallelizzazione anche della fase di *back substitution*, attualmente eseguita sulla CPU, valutando se il beneficio in termini di prestazioni giustifichi il costo di trasferimento dei dati.

Per quanto riguarda la versione p4, sarebbe interessante analizzare in modo più approfondito l'impatto del *dynamic parallelism*, valutando eventuali strategie per sfruttarlo in modo più efficiente al crescere della complessità dei dati.

Per quanto concerne la versione p5, sarebbe opportuno valutare eventuali strategie per ridurre l'overhead e renderla più efficiente per matrici di dimensioni ridotte.

Infine, un possibile sviluppo consiste nell'estendere l'analisi sperimentale, considerando:

- diverse dimensioni di input;
- differenti architetture GPU;
- variazioni nel numero di thread e blocchi.

al fine di valutare in modo più completo la scalabilità delle soluzioni proposte.

Appendice A

Istruzioni di compilazione e guida all'uso

Tutti i file descritti sono reperibili nella *repository* GitHub pubblica del progetto ([link](#))

A.1 Guida all'uso del Makefile

Il Makefile fornito consente di compilare ed eseguire in modo semplice tutte le versioni dell'algoritmo sviluppato, sia in modalità seriale che parallela. L'utilizzo è pensato per facilitare il *testing* e il confronto tra le diverse implementazioni.

A.1.1 Compilazione del progetto

Per compilare l'intero progetto è sufficiente eseguire il comando:

```
make main
```

Questo comando genera tutti gli eseguibili necessari, in particolare:

- *tester*, che permette di eseguire le varie versioni dell'algoritmo;
- *p_demo*, utilizzato per eseguire test su file di input;
- *SerialeDemo*, versione standalone dell'implementazione seriale.

A.1.2 Esecuzione delle versioni

Una volta compilato il progetto, è possibile eseguire le diverse versioni dell'algoritmo tramite specifici target.

Per eseguire la versione seriale si utilizza:

```
make run_seriale
```

Per eseguire una versione parallela specifica è disponibile un target parametrico:

```
make run_pX
```

dove ($X \in 1, 2, 3, 4, 5$). Ad esempio:

```
make run_p3
```

esegue la versione parallela p3.

È inoltre possibile eseguire più versioni in sequenza tramite:

```
make run_all
```

utile per effettuare confronti diretti tra le implementazioni più significative.

A.1.3 Esecuzione su file di test

Il Makefile consente anche di eseguire automaticamente i programmi su una serie di file di input predefiniti e di dimensione ridotta.

Per eseguire una versione parallela su tutti i file di test si utilizza:

```
make demo_pX
```

dove X indica la versione desiderata. Il comando esegue il programma su ciascun file di input, automatizzando il processo di *testing*.

Per la versione seriale è disponibile il comando:

```
make demo_ser
```

che esegue l'implementazione seriale sugli stessi file di test.

A.1.4 Pulizia dei file

Per rimuovere tutti i file generati durante la compilazione è possibile utilizzare:

```
make clean
```

Questo comando elimina file oggetto ed eseguibili, permettendo di ricompilare il progetto da zero.

A.2 File main.cu

Questo file implementa il programma principale utilizzato per eseguire i test sperimentali sulle diverse versioni dell'algoritmo di eliminazione di eliminazione Gauss per sistemi booleani.

Il programma riceve come input una stringa da linea di comando che identifica la versione dell'algoritmo da eseguire (seriale oppure una delle versioni parallele). In base a questa scelta, seleziona l'implementazione corrispondente e apre un file CSV dedicato in cui vengono salvati i risultati.

Il numero di test da eseguire viene definito con la macro **N**. Per ogni test, vengono generate matrici quadrate di dimensione crescente, con valori booleani distribuiti casualmente secondo un parametro θ , che controlla la densità della matrice. Per ciascuna dimensione, l'esecuzione viene ripetuta dieci volte (valore determinato dalla macro **REP**), così da ottenere misure di tempo più stabili.

A seconda della versione considerata, la matrice viene convertita nel formato richiesto: rappresentazione classica (booleani), array lineare oppure formato compatto (bit-packing su interi a 32 bit). Successivamente viene eseguito l'algoritmo e viene misurato il tempo di esecuzione.

Se abilitato, tramite la macro booleana **CONTROL** il programma verifica anche la correttezza della soluzione calcolata, controllando che tutte le equazioni del sistema siano soddisfatte. Questo passaggio è utile per garantire l'affidabilità dei risultati.

Infine, per ogni esecuzione vengono salvati nel file CSV la dimensione del problema, il parametro θ , il tempo di esecuzione, l'esito dell'algoritmo e il risultato del controllo.

A.3 file `matrixgenerator.c`

Lo scopo di questo file è di definire una funzione che permetta di generare una matrice di booleani in base al parametro di densità θ compreso tra 0 e 1. Viene quindi generata una matrice e per ogni cella la probabilità che il valore sia 1 è pari a θ . Viene fissato un *seed* per permettere le comparazioni dei test tra le diverse configurazioni e versioni.

A.4 file `Analysis.R`

In questo file è presente il codice per la produzione di tabelle, visualizzazioni e verificare la correttezza dei risultati confrontandoli con quelli della versione seriale.

A.5 file `SerialeDemo.c` e `p_demo.cu`

Questi due file servono per eseguire gli algoritmi su test di dimensioni molto ridotte a solo scopo didattico ed illustrativo.

A.6 directory `result_data` e `test`

Nella cartella `result_data` è possibile trovare i file csv contenenti tutti i tempi di esecuzione registrati dei vari test. Il nome di un generico file csv è `"risultati_NomeVersione_theta_N_GPUutilizzata.csv"`. Nella cartella `test` sono presenti dei test case molto semplici utilizzabili con i comandi `make demo_ser` e `make demo_pX`.